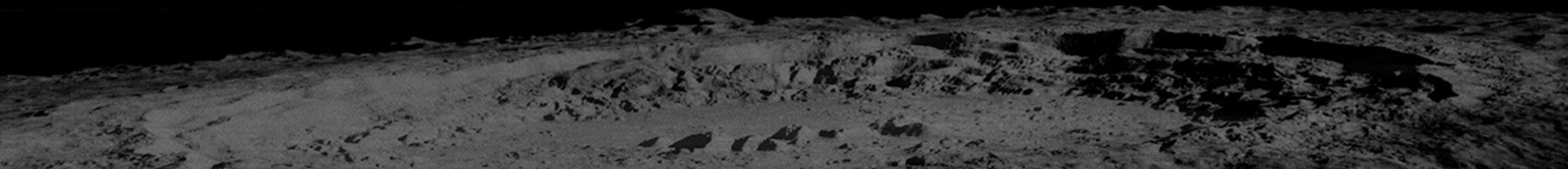




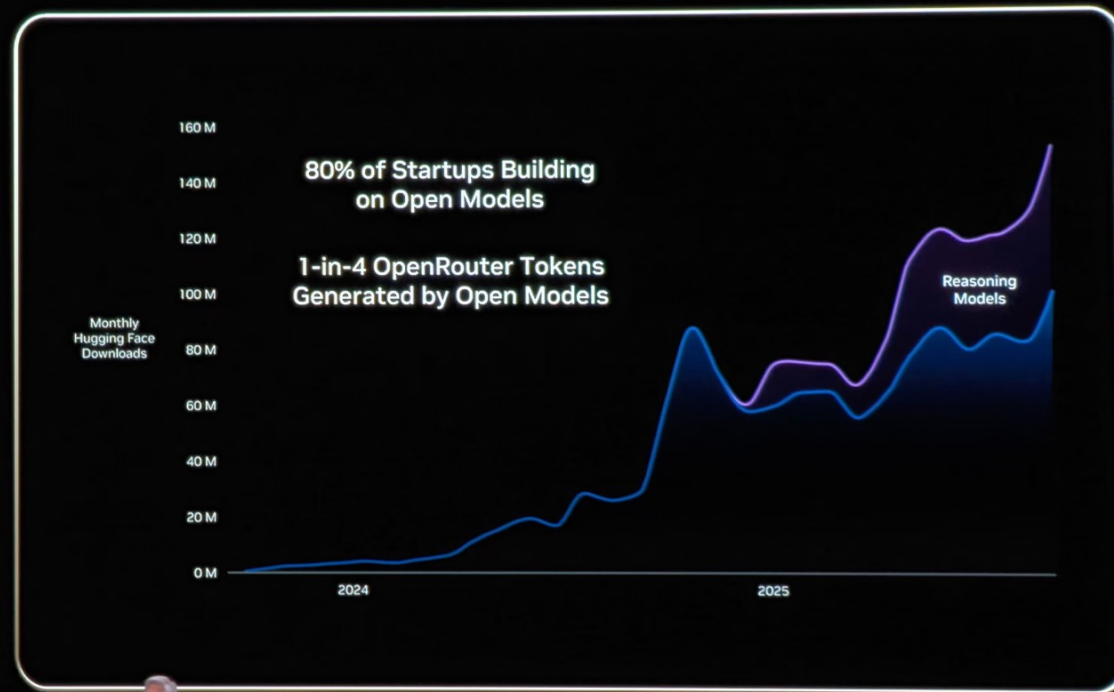
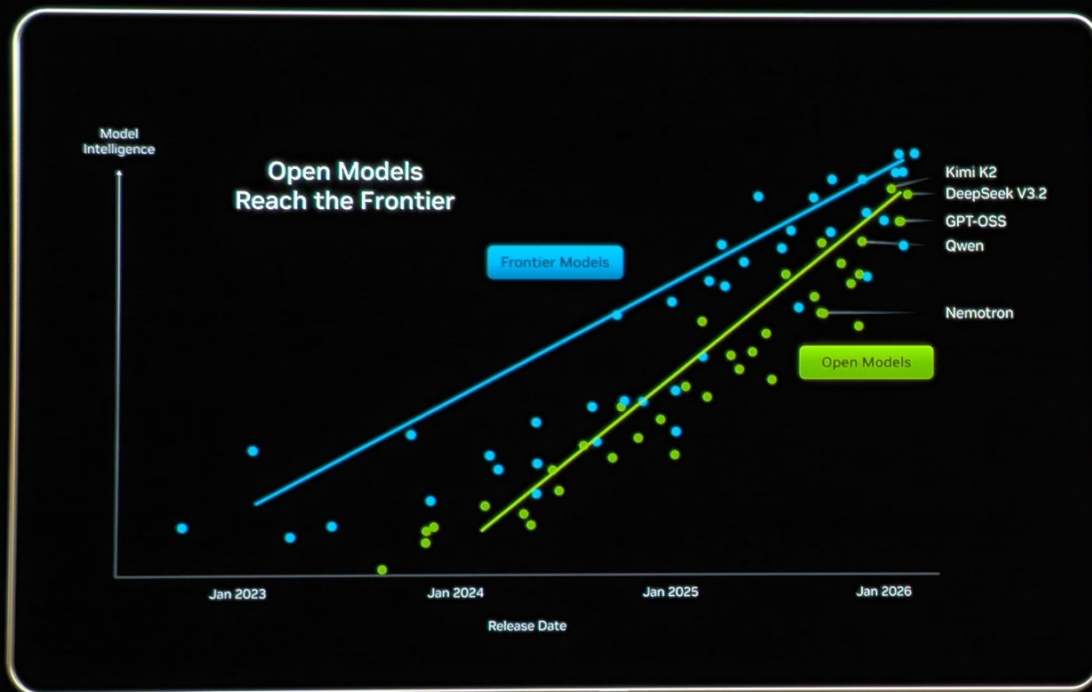
How We Scaled Kimi K2.5

Zhilin Yang

Founder and CEO of Kimi

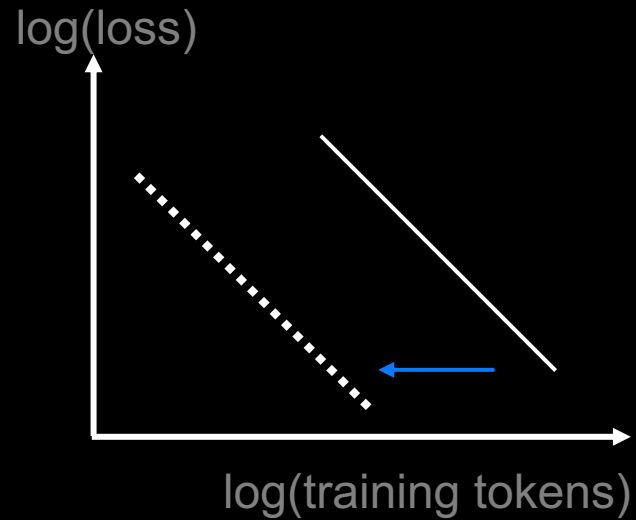


Kimi's Open Models: Democratizing Intelligence

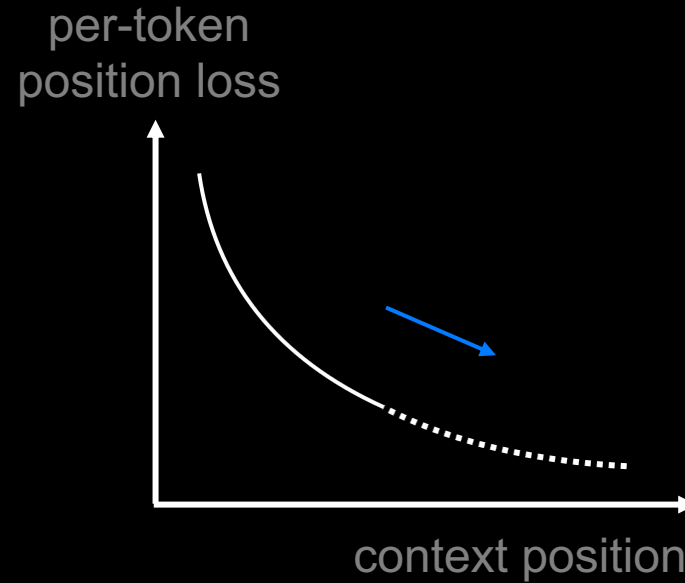


Kimi's Scaling Strategy

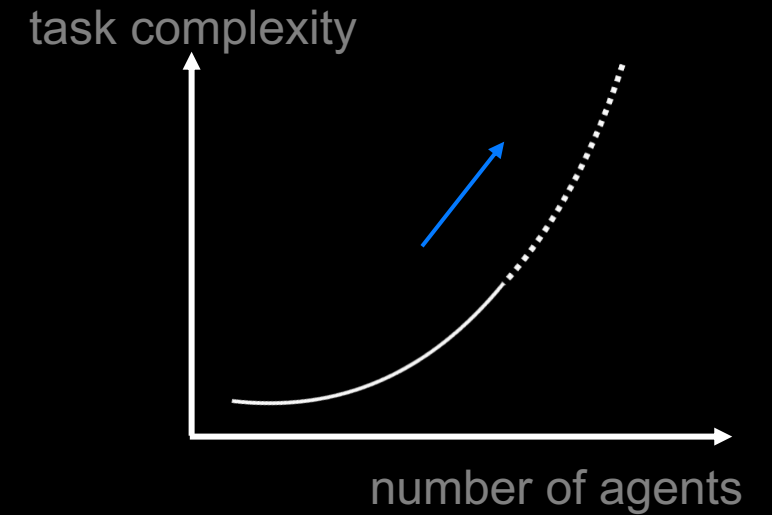
Token Efficiency



Long Context



Agent Swarms





Token Efficiency

Scaling Law: The First Principles of LLMs

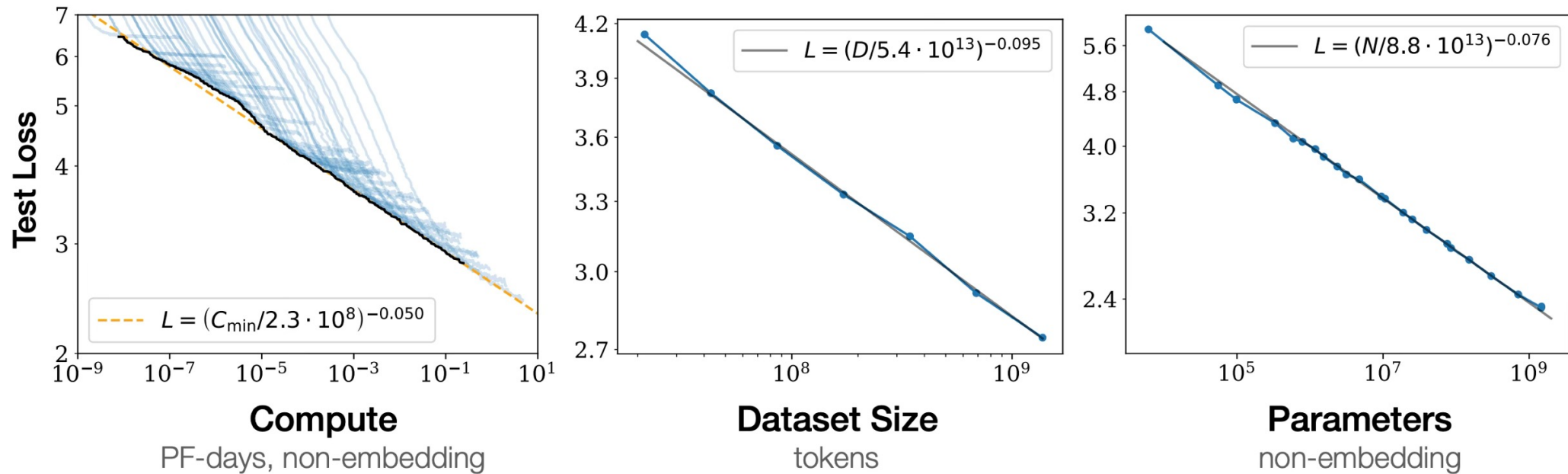
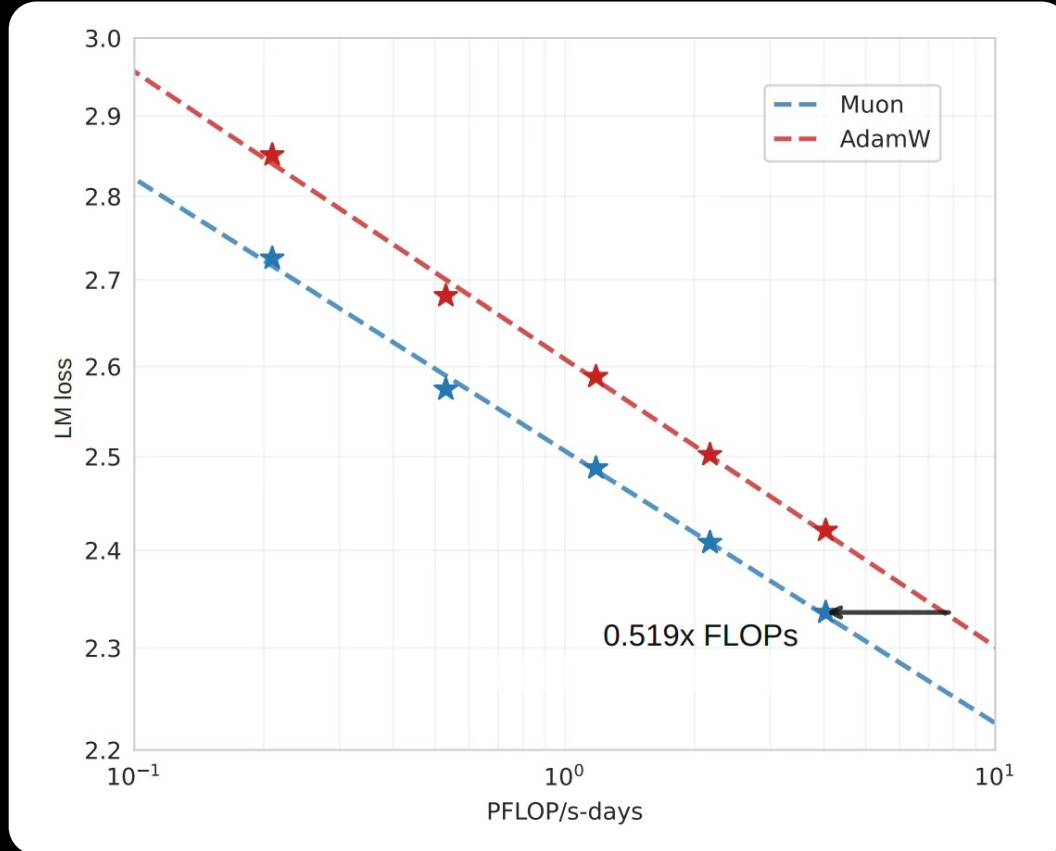


Figure 1 Language modeling performance improves smoothly as we increase the model size, dataset size, and amount of compute² used for training. For optimal performance all three factors must be scaled up in tandem. Empirical performance has a power-law relationship with each individual factor when not bottlenecked by the other two.

Token Efficiency



Muon:
~2x token efficiency



Muon is Scalable for LLM Training

μ MUON IS SCALABLE FOR LLM TRAINING

TECHNICAL REPORT

Jingyuan Liu¹ Jianlin Su¹ Xingcheng Yao² Zhejun Jiang¹ Guokun Lai¹ Yulun Du¹
Yidao Qin¹ Weixin Xu¹ Enzhe Lu¹ Junjie Yan¹ Yanru Chen¹ Huabin Zheng¹
Yibo Liu¹ Shaowei Liu¹ Bohong Yin¹ Weiran He¹ Han Zhu¹ Yuzhi Wang¹
Jianzhou Wang¹ Mengnan Dong¹ Zheng Zhang¹ Yongsheng Kang¹ Hao Zhang¹
Xinran Xu¹ Yutao Zhang¹ Yuxin Wu¹ Xinyu Zhou¹ Zhilin Yang¹

¹ Moonshot AI ² UCLA

ABSTRACT

Recently, the Muon optimizer (K. Jordan et al. 2024) based on matrix orthogonalization has demonstrated strong results in training small-scale language models, but the scalability to larger models has not been proven. We identify two crucial techniques for scaling up Muon: (1) adding weight decay and (2) carefully adjusting the per-parameter update scale. These techniques allow Muon to work out-of-the-box on large-scale training without the need of hyper-parameter tuning. Scaling law experiments indicate that Muon achieves $\sim 2\times$ computational efficiency compared to AdamW with compute optimal training. Based on these improvements, we introduce Moonlight, a 3B/16B-parameter Mixture-of-Expert (MoE) model trained with 5.7T tokens using Muon. Our model improves the current Pareto frontier, achieving better performance with much fewer training FLOPs compared to prior models. We open-source our distributed Muon implementation that is memory optimal and communication efficient. We also release the pretrained, instruction-tuned, and intermediate checkpoints to support future research.

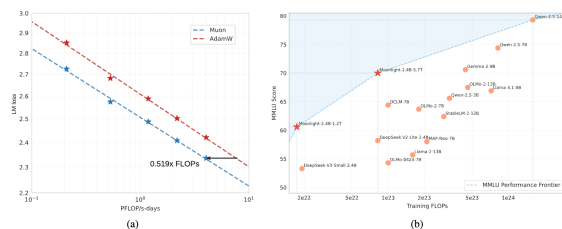


Figure 1: Scaling up with Muon. (a) Scaling law experiments comparing Muon and Adam. Muon is $\sim 2\times$ more computationally efficient than Adam with compute optimal training. (b) The MMLU performance of our Moonlight model optimized with Muon and other comparable models. Moonlight advances the Pareto frontier of performance vs training FLOPs.

*Corresponding author: zhouxinyu@moonshot.cn

Scale Muon Optimizer

- **Weight Decay: Critical for scaling to larger models**
- **Consistent RMS Updates: Fine-tuning per-parameter update magnitudes, enforcing consistent root-mean-square**

Scaling Law: Experimental Results

- **$\sim 2\times$ token efficiency (vs AdamW)**

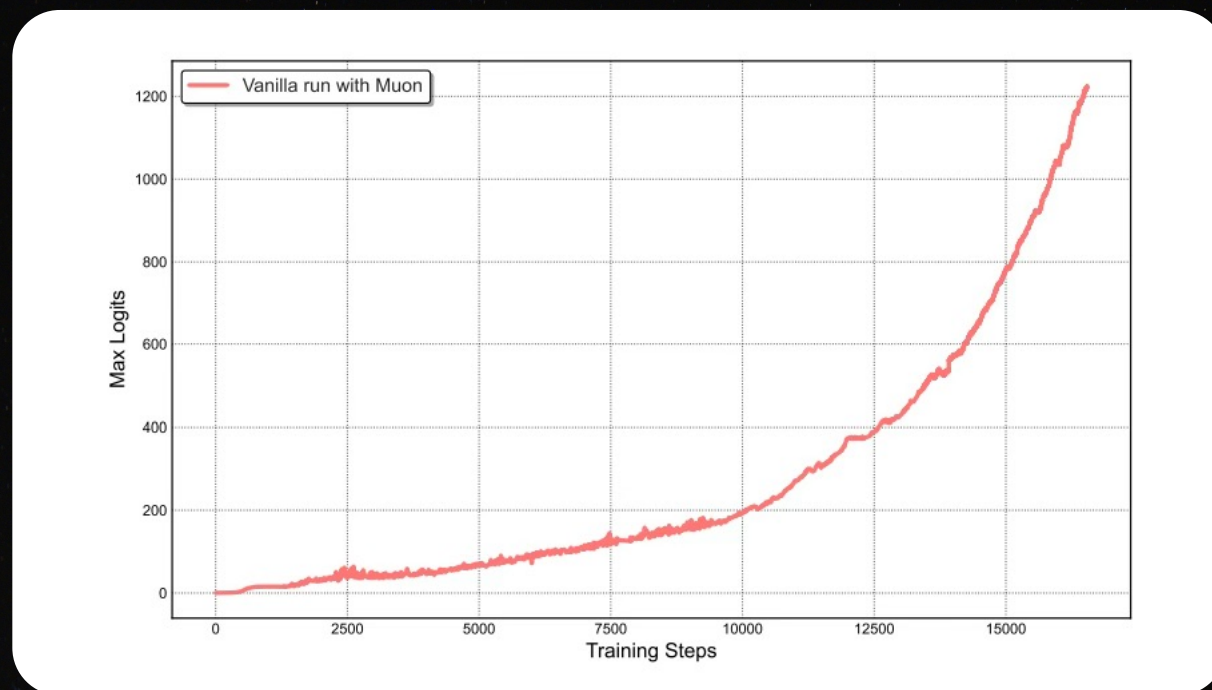
To make Muon memory-efficient across the NVIDIA GPU cluster, engineers designed a “Distributed Muon” algorithm that partitions states across the Data Parallel (DP) group, gathering the gradients just in-time to perform Newton-Schulz iterations.

Muon is Scalable for LLM Training

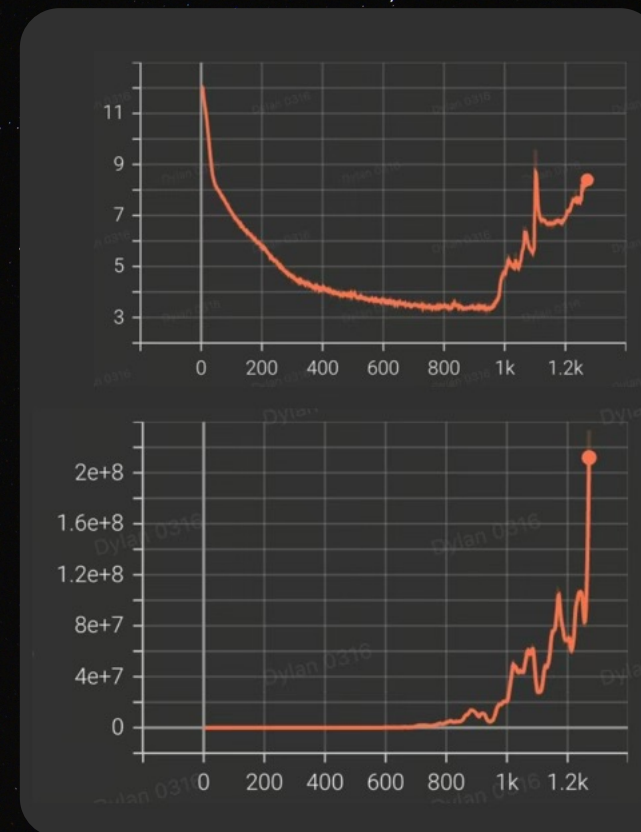
Table 5: Comparison of different models on various benchmarks.

Benchmark (Metric)		Llama3.2-3B	Qwen2.5-3B	DSV2-Lite	Moonlight
	Activated Param [†]	2.81B	2.77B	2.24B	2.24B
	Total Params [†]	2.81B	2.77B	15.29B	15.29B
	Training Tokens	9T	18T	5.7T	5.7T
	Optimizer	AdamW	Unknown	AdamW	Muon
English	MMLU	54.7	65.6	58.3	70.0
	MMLU-pro	25.0	34.6	25.5	42.4
	BBH	46.8	56.3	44.1	65.2
	TriviaQA [‡]	59.6	51.1	65.1	66.3
Code	HumanEval	28.0	42.1	29.9	48.1
	MBPP	48.7	57.1	43.2	63.8
Math	GSM8K	34.0	79.1	41.1	77.4
	MATH	8.5	42.6	17.1	45.3
	CMATH	-	80.0	58.4	81.1
Chinese	C-Eval	-	75.0	60.3	77.2
	CMMLU	-	75.0	64.3	78.2

Training Instabilities When Scaling Muon



Mid-scale Muon training: max logits quickly exceeds 1,000



An example shows max logits explosions cause divergence

Taming Muon with QK-Clip

Algorithm 1 MuonClip Optimizer

```

1: for each training step  $t$  do
2:   // 1. Muon optimizer step
3:   for each weight  $\mathbf{W} \in \mathbb{R}^{n \times m}$  do
4:      $\mathbf{M}_t = \mu \mathbf{M}_{t-1} + \mathbf{G}_t$  ▷  $\mathbf{M}_0 = \mathbf{0}$ ,  $\mathbf{G}_t$  is the grad of  $\mathbf{W}_t$ ,  $\mu$  is momentum
5:      $\mathbf{O}_t = \text{Newton-Schulz}(\mathbf{M}_t) \cdot \sqrt{\max(n, m)} \cdot 0.2$  ▷ Match Adam RMS
6:      $\mathbf{W}_t = \mathbf{W}_{t-1} - \eta(\mathbf{O}_t + \lambda \mathbf{W}_{t-1})$  ▷ learning rate  $\eta$ , weight decay  $\lambda$ 
7:   end for
8:   // 2. QK-Clip
9:   for each attention head  $h$  in every attention layer of the model do
10:    Obtain  $S_{\max}^h$  already computed during forward
11:    if  $S_{\max}^h > \tau$  then
12:       $\gamma \leftarrow \tau / S_{\max}^h$ 
13:       $\mathbf{W}_{qc}^h \leftarrow \mathbf{W}_{qc}^h \cdot \sqrt{\gamma}$  •  $\mathbf{q}^C$  and  $\mathbf{k}^C$  (head-specific components): each scaled by  $\sqrt{\gamma_h}$ 
14:       $\mathbf{W}_{kc}^h \leftarrow \mathbf{W}_{kc}^h \cdot \sqrt{\gamma}$  •  $\mathbf{q}^R$  (head-specific rotary): scaled by  $\gamma_h$ ,
15:       $\mathbf{W}_{qr}^h \leftarrow \mathbf{W}_{qr}^h \cdot \gamma$  •  $\mathbf{k}^R$  (shared rotary): left untouched to avoid effect across heads.
16:    end if
17:  end for
18: end for

```

$$\mathbf{Q}^h = \mathbf{XW}_q^h, \quad \mathbf{K}^h = \mathbf{XW}_k^h, \quad \mathbf{V}^h = \mathbf{XW}_v^h.$$

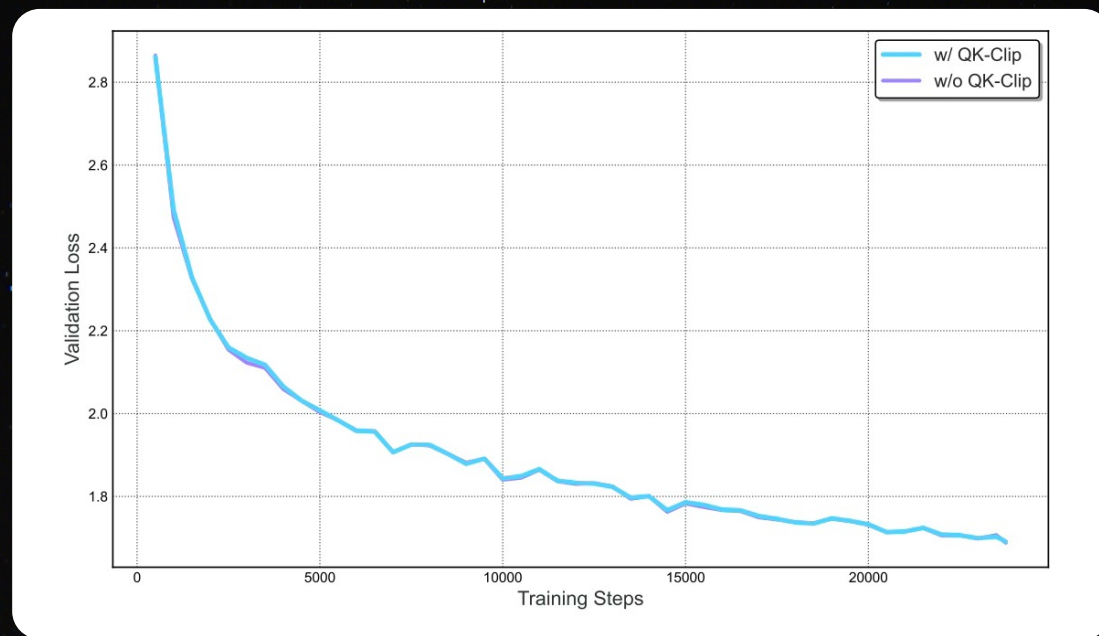
$$\mathbf{O}^h = \text{softmax} \left(\frac{1}{\sqrt{d}} \mathbf{Q}^h \mathbf{K}^{h\top} \right) \mathbf{V}^h.$$

$$S_{\max}^h = \frac{1}{\sqrt{d}} \max_{\mathbf{X} \in B} \max_{i,j} \mathbf{Q}_i^h \mathbf{K}_j^{h\top}$$

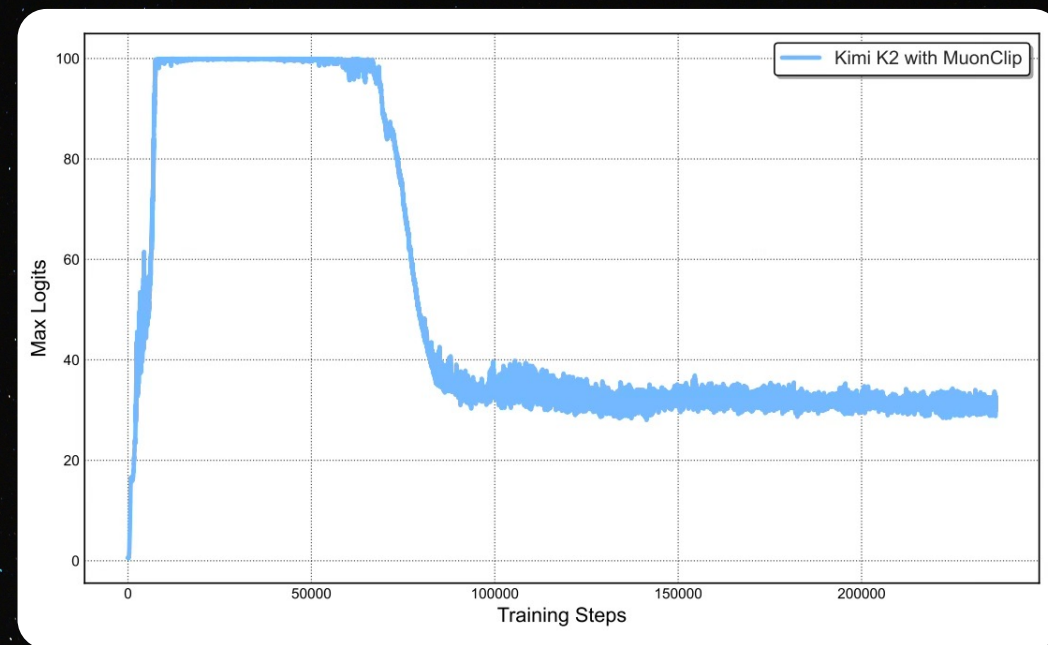
$$\mathbf{W}_q^h \leftarrow \gamma^\alpha \mathbf{W}_q^h \quad \mathbf{W}_k^h \leftarrow \gamma^{1-\alpha} \mathbf{W}_k^h$$

where $\gamma = \min(1, \tau / S_{\max})$ with $S_{\max} = \max_h S_{\max}^h$.

Stable Training with MuonClip



Small-scale MuonClip validation: No harm on loss



Kimi K2 with MuonClip: max logits <100 and eventually decays



**Long
Context**

Kaplan Law's Hidden Gem

Transformers vs LSTMs

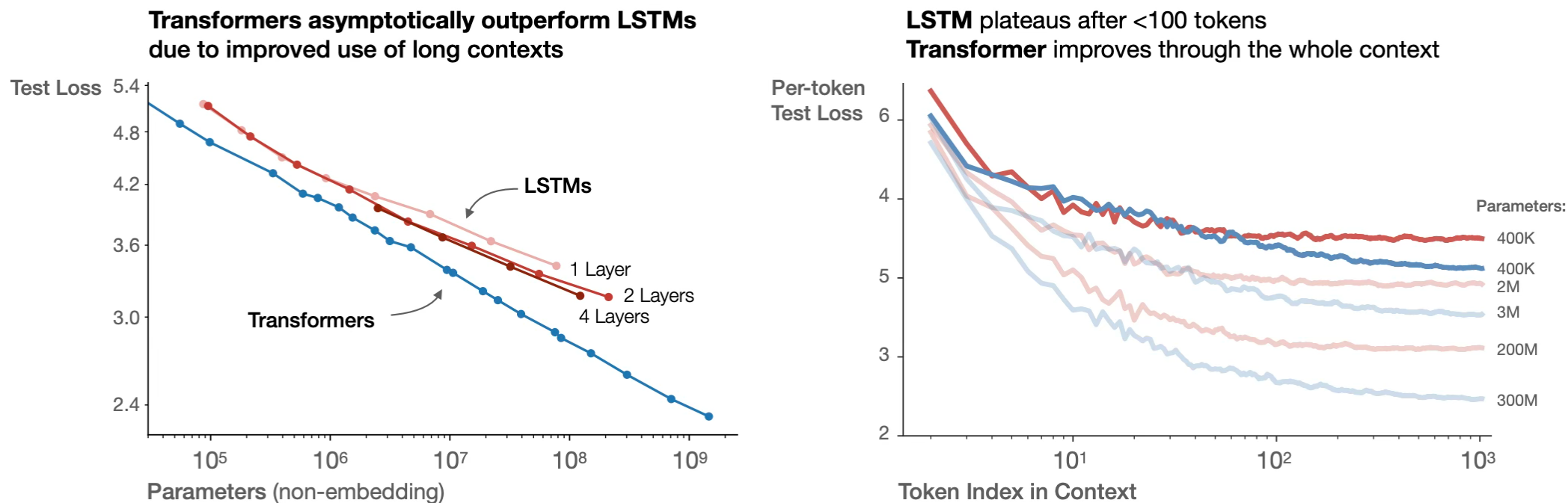


Figure 7

Kimi Linear

KIMI LINEAR: AN EXPRESSIVE, EFFICIENT ATTENTION ARCHITECTURE

TECHNICAL REPORT OF KIMI LINEAR

Kimi Team

<https://github.com/MoonshotAI/Kimi-Linear>

ABSTRACT

We introduce Kimi Linear, a hybrid linear attention architecture that, for the first time, outperforms full attention under fair comparisons across various scenarios—including short-context, long-context, and reinforcement learning (RL) scaling regimes. At its core lies Kimi Delta Attention (KDA), an expressive linear attention module that extends Gated DeltaNet [111] with a finer-grained gating mechanism, enabling more effective use of limited finite-state RNN memory. Our bespoke chunkwise algorithm achieves high hardware efficiency through a specialized variant of the *Diagonal-Plus-Low-Rank* (DPLR) transition matrices, which substantially reduces computation compared to the general DPLR formulation while remaining more consistent with the classical delta rule.

We pretrain a Kimi Linear model with 3B activated parameters and 48B total parameters, based on a layerwise hybrid of KDA and Multi-Head Latent Attention (MLA). Our experiments show that with an identical training recipe, Kimi Linear outperforms full MLA with a sizeable margin across all evaluated tasks, while reducing KV cache usage by up to 75% and achieving up to 6× decoding throughput for a 1M context. These results demonstrate that Kimi Linear can be a drop-in replacement for full attention architectures with superior performance and efficiency, including tasks with longer input and output lengths.

To support further research, we open-source the KDA kernel and vLLM implementations¹, and release the pre-trained and instruction-tuned model checkpoints.²

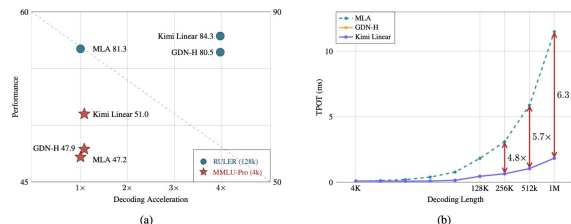


Figure 1: (a) Performance vs. acceleration. With strict fair comparisons with 1.4T training tokens, on MMLU-Pro (4k context length, red stars), Kimi Linear leads performance (51.0) at similar speed. On RULER (128k context length, blue circles), it is Pareto-optimal, achieving top performance (84.3) and 3.98× acceleration. (b) Time per output token (TPOT) vs. decoding length. Kimi Linear (blue line) maintains a low TPOT, matching GDN-H and outperforming MLA at long sequences. This enables larger batches, yielding a 6.3× faster TPOT (1.84ms vs. 11.48ms) than MLA at 1M tokens.

¹<https://github.com/flax-org/flax-linear-attention/tree/main/flax/ops/kda>

²<https://huggingface.co/moonshotai/Kimi-Linear-48B-A3B-Instruct>

- **Kimi Delta Attention (KDA):** Linear attention mechanism optimizing Gated Delta Rule via improved recurrent memory management and hardware efficiency.
- **Kimi Linear Architecture:** Hybrid design with 3:1 KDA-to-global attention ratio; cuts memory overhead while outperforming full attention.
- **Large-Scale Empirical Validation:** 1.4T token training shows Kimi Linear beats full attention and baselines on long/short context and RL tasks. Fully open-sourced: kernels, vLLM integration, and model weights.

Kimi Linear: An Expressive, Efficient Attention Architecture

$$\mathbf{S}_t = (\mathbf{I} - \beta_t \mathbf{k}_t \mathbf{k}_t^\top) \text{Diag}(\boldsymbol{\alpha}_t) \mathbf{S}_{t-1} + \beta_t \mathbf{k}_t \mathbf{v}_t^\top \in \mathbb{R}^{d_k \times d_v}; \quad \mathbf{o}_t = \mathbf{S}_t^\top \mathbf{q}_t \in \mathbb{R}^{d_v}$$

$$\begin{bmatrix} \text{blue grid} \end{bmatrix} = \left(\begin{bmatrix} \text{grey grid} \end{bmatrix} - \begin{bmatrix} \text{orange column} \end{bmatrix} \times \begin{bmatrix} \text{orange row} \end{bmatrix} \right) \begin{bmatrix} \text{red grid} \end{bmatrix} + \begin{bmatrix} \text{orange column} \end{bmatrix} \times \begin{bmatrix} \text{orange row} \end{bmatrix} \quad \Bigg| \quad \begin{bmatrix} \text{orange column} \end{bmatrix} = \begin{bmatrix} \text{blue grid} \end{bmatrix} \begin{bmatrix} \text{orange column} \end{bmatrix}$$

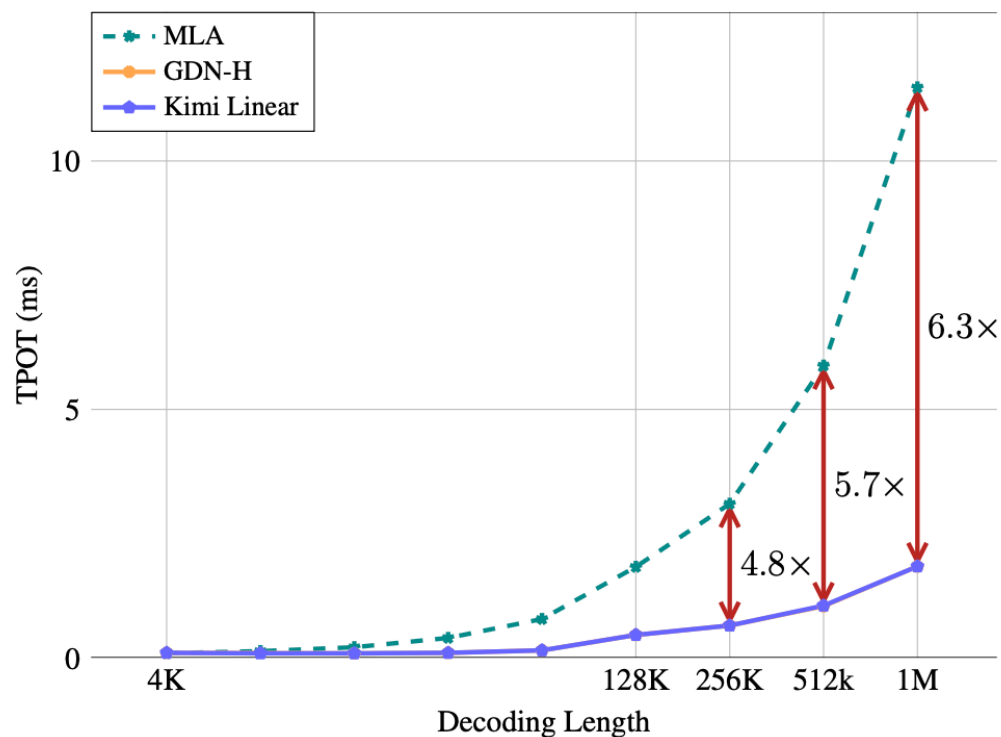
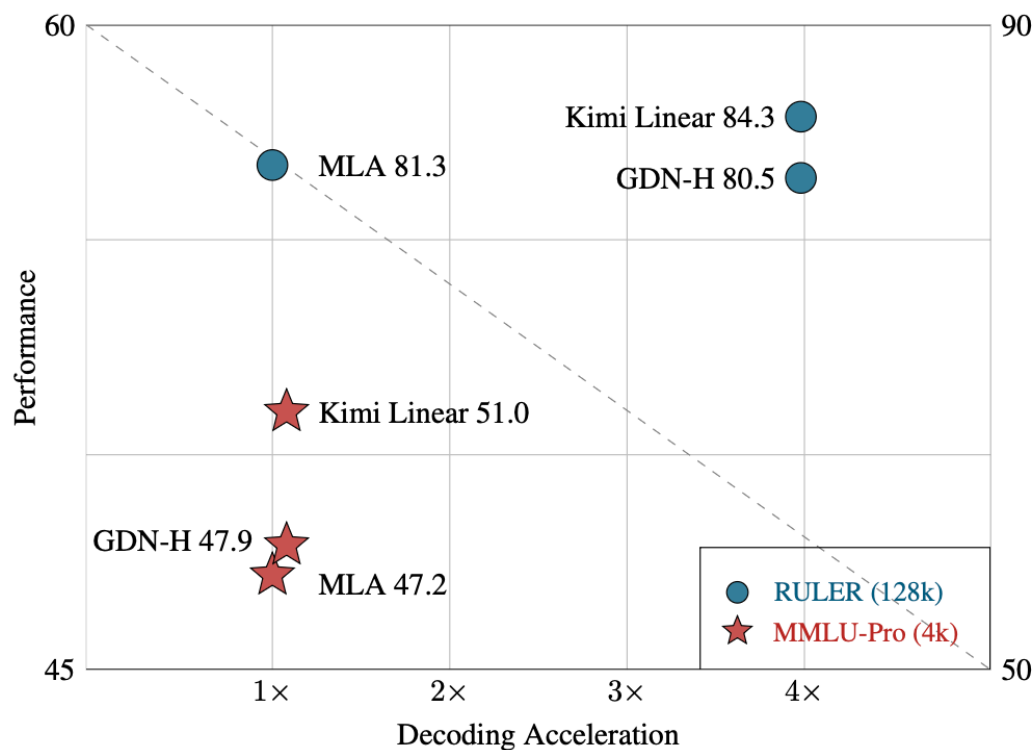
Kimi Linear: An Expressive, Efficient Attention Architecture

$$\mathbf{S}_{[t]}^r = \underbrace{\left(\prod_{i=1}^r \left(\mathbf{I} - \beta_{[t]}^i \mathbf{k}_{[t]}^i \mathbf{k}_{[t]}^{i\top} \right) \text{Diag}(\boldsymbol{\alpha}_{[t]}^i) \right)}_{:= \mathbf{P}_{[t]}^r} \cdot \mathbf{S}_{[t]}^0 + \underbrace{\sum_{i=1}^r \left(\prod_{j=i+1}^r \left(\mathbf{I} - \beta_{[t]}^j \mathbf{k}_{[t]}^j \mathbf{k}_{[t]}^{j\top} \right) \text{Diag}(\boldsymbol{\alpha}_{[t]}^j) \right)}_{:= \mathbf{H}_{[t]}^r} \cdot \beta_{[t]}^i \mathbf{k}_{[t]}^i \mathbf{v}_{[t]}^{i\top}$$

- **Efficient Implementation:** More efficient than DPLR, more expressive than GDN

$$\begin{aligned} \mathbf{S}_{[t+1]} &= \text{Diag}(\boldsymbol{\gamma}_{[t]}^C) \mathbf{S}_{[t]} + \left(\boldsymbol{\Gamma}_{[t]}^{i \rightarrow C} \odot \mathbf{K}_{[t]} \right)^\top (\mathbf{U}_{[t]} - \mathbf{W}_{[t]} \mathbf{S}_{[t]}) \in \mathbb{R}^{d_k \times d_v} \\ \mathbf{M}_{[t]} &= \left(\mathbf{I} + \text{StrictTril} \left(\text{Diag}(\beta_{[t]}) \left(\boldsymbol{\Gamma}_{[t]}^{1 \rightarrow C} \odot \mathbf{K}_{[t]} \right) \left(\frac{\mathbf{K}_{[t]}}{\boldsymbol{\Gamma}_{[t]}^{1 \rightarrow C}} \right)^\top \right) \right)^{-1} \text{Diag}(\beta_{[t]}) \\ \mathbf{W}_{[t]} &= \mathbf{M}_{[t]} \left(\boldsymbol{\Gamma}_{[t]}^{1 \rightarrow C} \odot \mathbf{K}_{[t]} \right), \quad \mathbf{U}_{[t]} = \mathbf{M}_{[t]} \mathbf{V}_{[t]} \end{aligned}$$

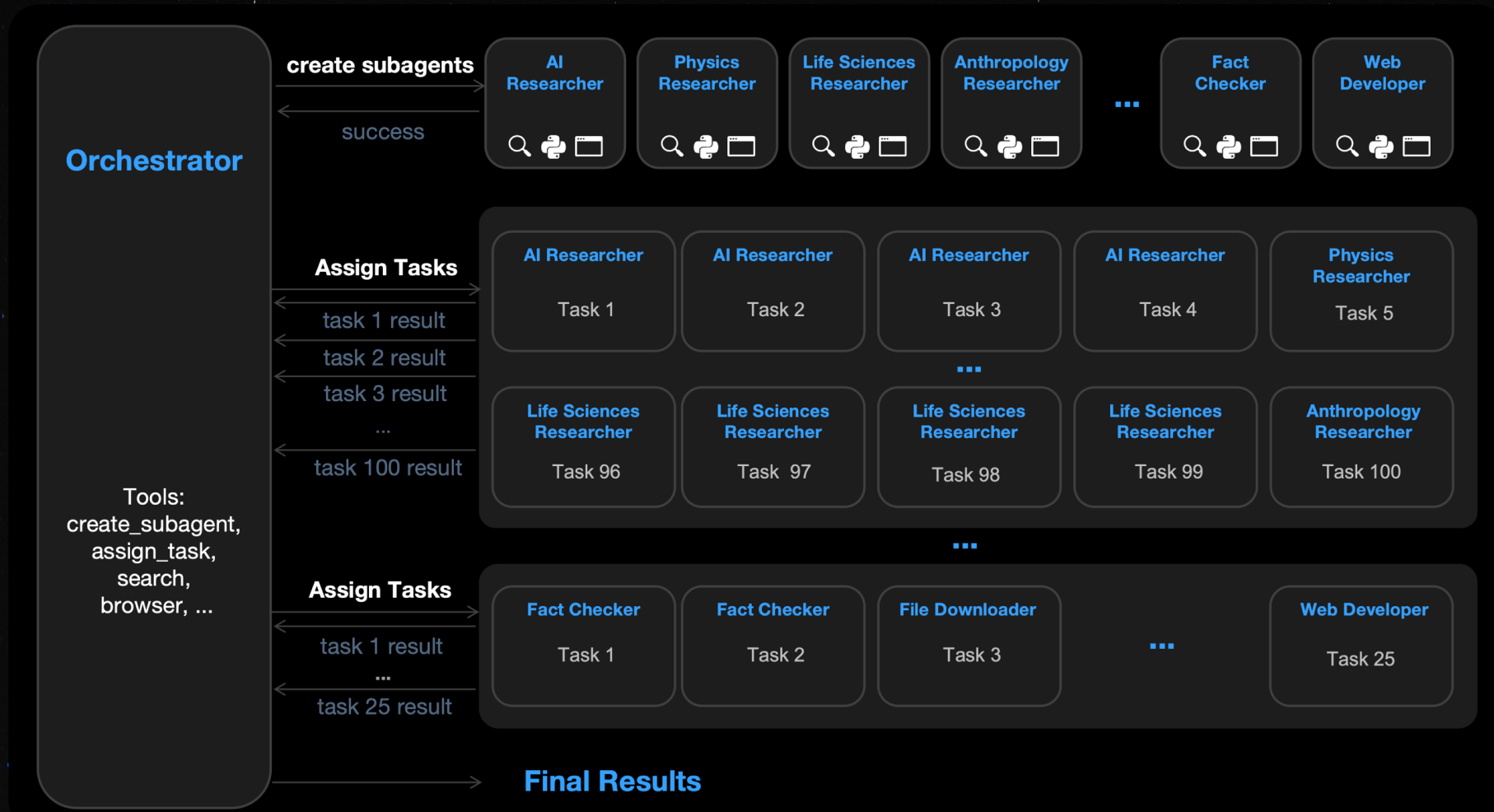
Kimi Linear: An Expressive, Efficient Attention Architecture





Agent Swarms

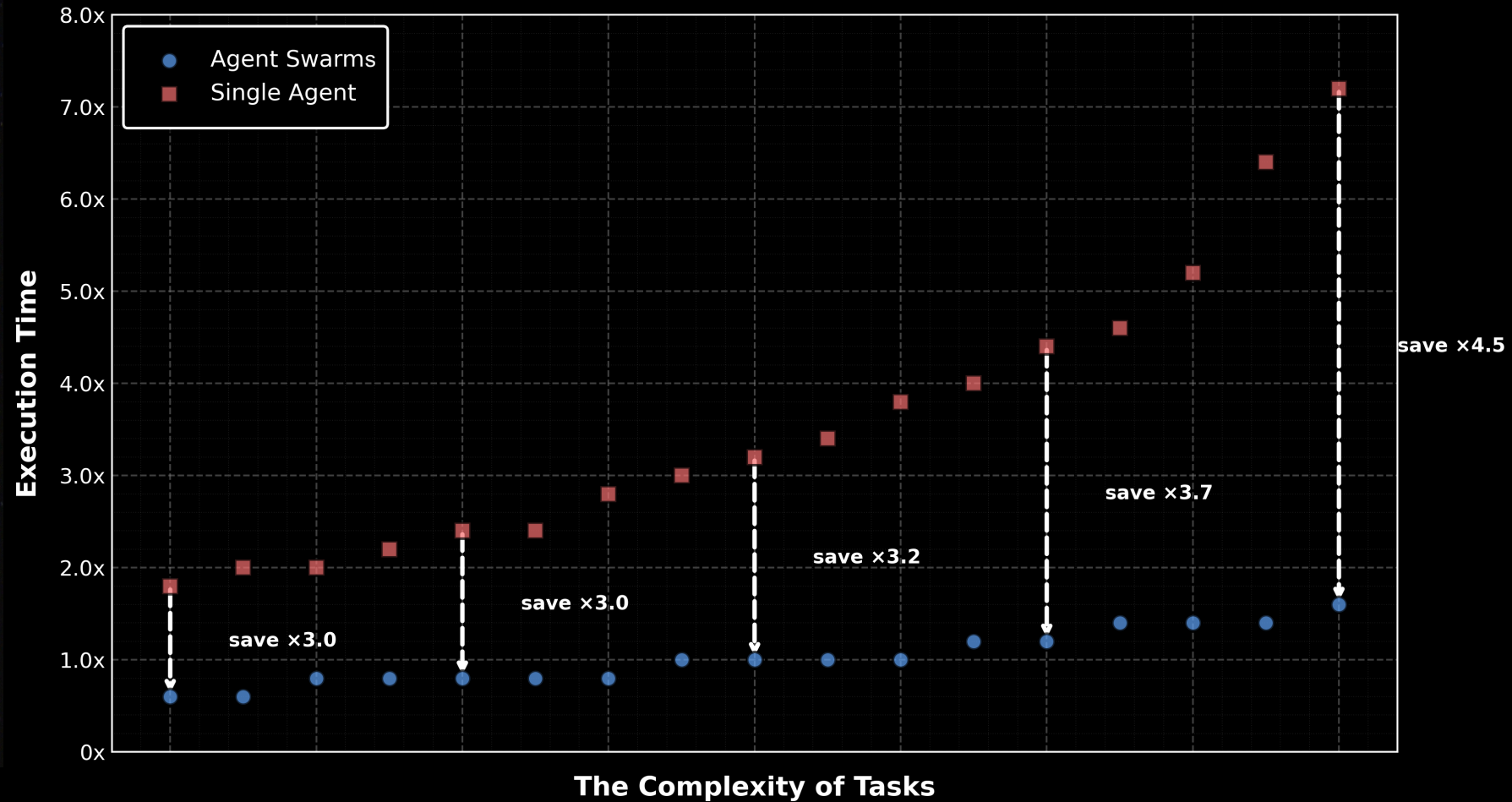
Agent Swarms



Agent swarms have an orchestrator that dynamically creates specialized subagents (e.g., AI Researcher, Physics Researcher, Fact Checker) and decomposes complex tasks into parallelizable subtasks for efficient distributed execution.

Agent Swarms

Execution Time to Solve Tasks of Different Complexity

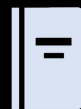


Agent Swarms



Input at Scale

Downloading and reading hundreds of sources in parallel



Output at Scale

Writing a 100-page literature review



Actions at Scale

Parallel data analysis jobs using Python



Orchestration at Scale

Designing subtasks and aggregating results

Reinforcement Learning for Agent Swarms

$$r_{\text{PARL}}(x, y) = \lambda_1 \cdot \underbrace{r_{\text{parallel}}}_{\text{instantiation reward}} + \lambda_2 \cdot \underbrace{r_{\text{finish}}}_{\text{sub-agent finish rate}} + \underbrace{r_{\text{perf}}(x, y)}_{\text{task-level outcome}} .$$

- **Instantiation reward:** incentivizes subagent instantiation to mitigate *serial collapse* which defaults to single-agent execution.
- **Finish reward:** encourages the completion of subtasks assigned to subagents. It is used to prevent *spurious parallelism*, where subagents are spawned without meaningful task decomposition.
- **Outcome reward:** whether the agent swarm accomplishes the end goal.

Kimi's Scaling Strategy



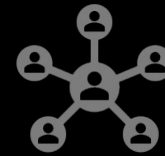
**Token
Efficiency**

MuonClip Optimizer



**Long
Context**

Kimi Delta Attention



**Agent
Swarms**

Agent Swarms

Vision



Coding

```
<!DOCTYPE  
<head><met  
charset="U  
le>My Kimi  
Page</titl  
ody(font-f  
le-system;  
stemFont,  
UI", Roboto  
f;display:
```

Meet Kimi K2.5

Open Visual Agentic Intelligence

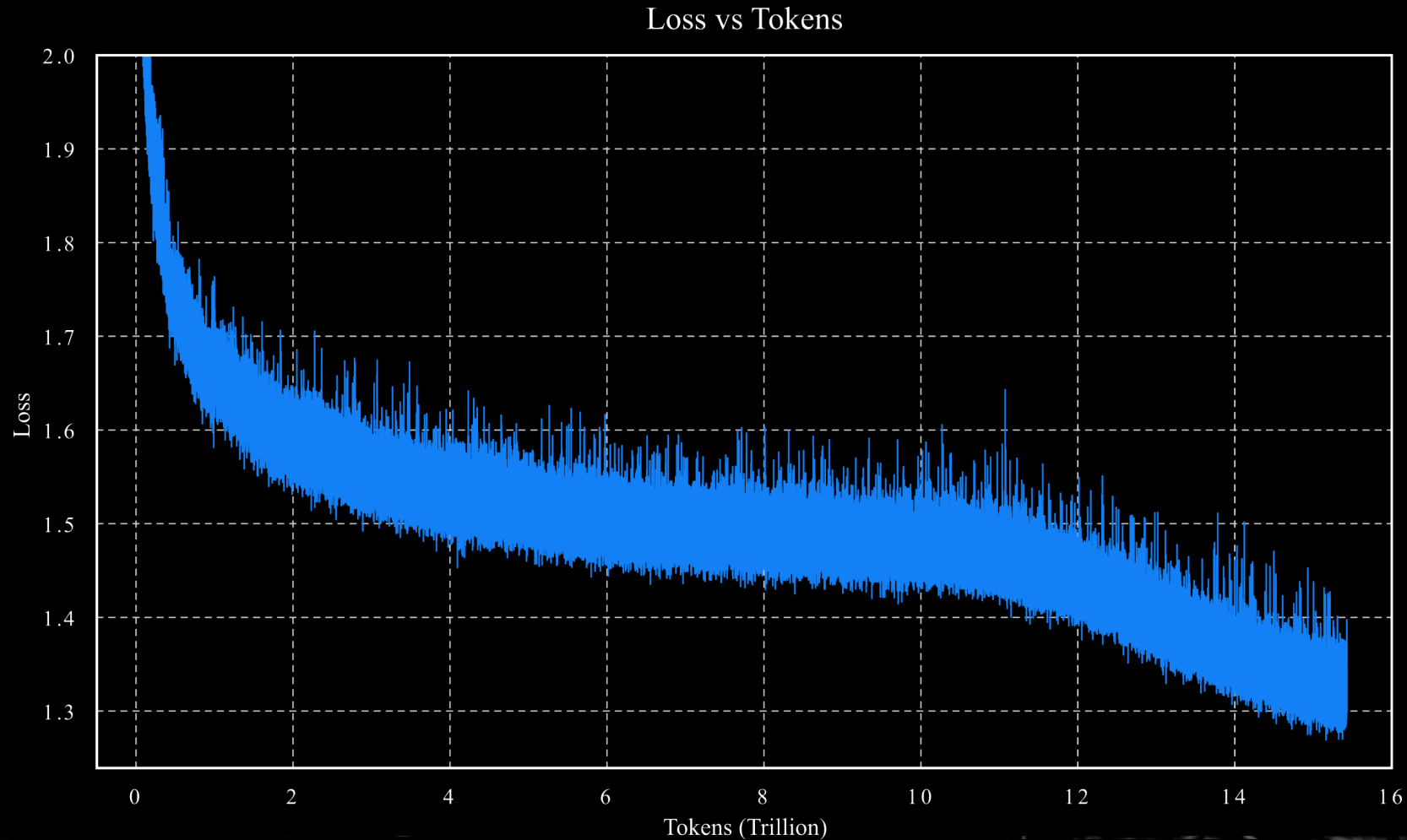


Agentic



KIMI

A Big and Beautiful Model



Kimi K2 and K2.5 were trained on a cluster equipped with NVIDIA H800 GPUs. Each node in the H800 cluster contains 2 TB RAM and 8 GPUs connected by NVLink and NVSwitch within nodes.

Kimi K2.5: Vision-Text Joint Pretraining

Table 1: Performance comparison across different vision-text joint-training strategies. Early fusion with a lower vision ratio yields better results given a fixed total vision-text token budget.

	Vision Injection Timing	Vision-Text Ratio	Vision Knowledge	Vision Reasoning	OCR	Text Knowledge	Text Reasoning	Code
Early	0%	10%:90%	25.8	43.8	65.7	45.5	58.5	24.8
Mid	50%	20%:80%	25.0	40.7	64.1	43.9	58.6	24.0
Late	80%	50%:50%	24.2	39.0	61.5	43.1	57.8	24.0

First open model with native, joint vision-text capabilities.

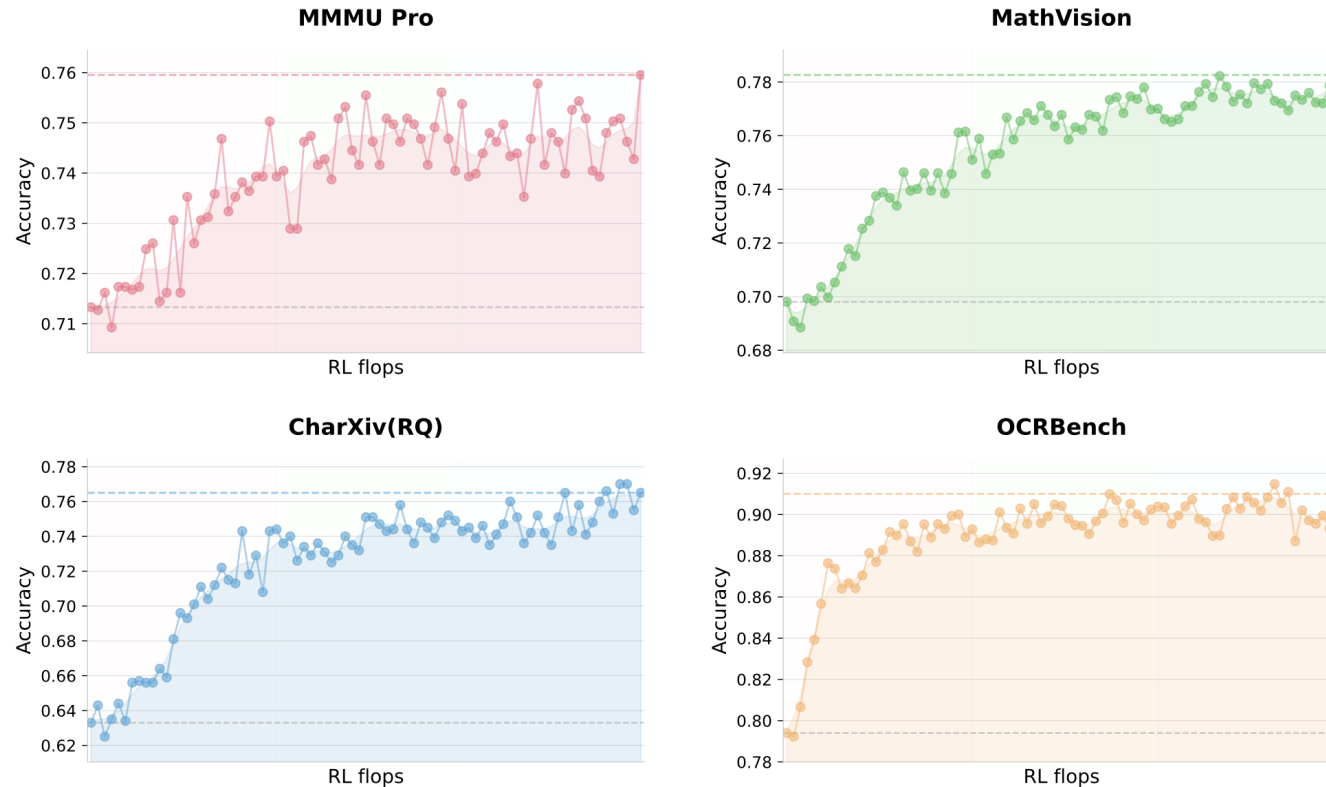
Vision Improves Text

Table 2: Cross-Modal Transfer: Vision RL Improves Textual Knowledge

Benchmark	Before Vision-RL	After Vision-RL	Improvement
MMLU-Pro	84.7	86.4	+1.7
GPQA-Diamond	84.3	86.4	+2.1
LongBench v2	56.7	58.9	+2.2

Visual RL Improves Text Performance

Zero-Vision SFT: Text Improves Vision



Vision RL training curves on vision benchmarks starting from minimal zero-vision SFT. Compared to zero-vision SFT, our preliminary experiments show that text-vision SFT yields much worse performance on visual, agentic tasks.

MARCUS

CONTEMPORARY SCULPTURE



EST. 1987



SCROLL TO ENTER

STEEL

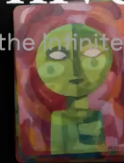


SCROLL TO ENTER



Kimi Universe

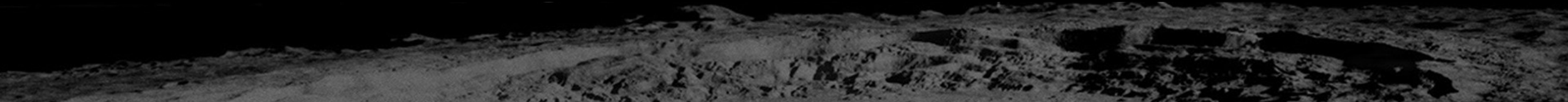
Explore the Infinite



SCROLL TO EXPLORE

One more thing...

A sneak peek at Kimi's next-gen arch

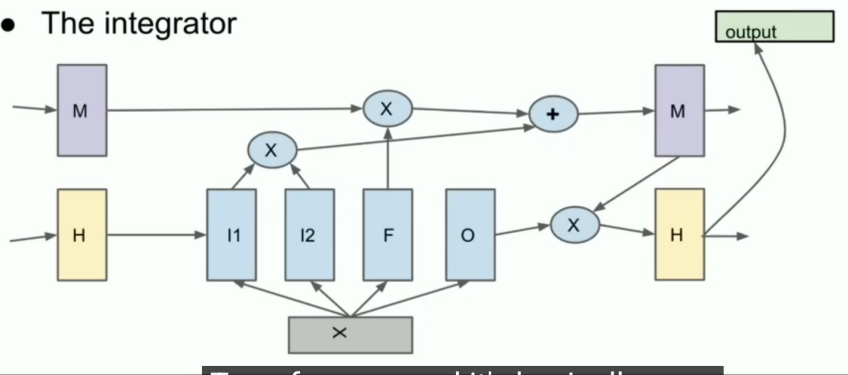


Ilya Sutskever: An LSTM is a ResNet rotated 90°

What we got wrong: the LSTM

The heart of the LSTM

- The integrator



Transformers and it's basically a res net but rotated 90° so that's

Rotating an LSTM gives us Residuals.

- Residual connections can be viewed as a simplified LSTM unrolled over depth.
- In this view, residuals provide only a narrow information highway through fixed addition.

What, then, is Attention rotated by 90°?

- Attention enriches this pathway by letting each layer retrieve broader, input-dependent representations.
- From this perspective, Attention rotated by 90° is a natural generalization of residual connections.

Introducing Attention Residuals

K ATTENTION RESIDUALS

TECHNICAL REPORT OF ATTENTION RESIDUALS

Kimi Team

<https://github.com/MoonshotAI/Attention-Residuals>

ABSTRACT

Residual connections [11] with PreNorm [55] are standard in modern LLMs, yet their fixed unit weights accumulate all layer outputs uniformly, leading to unbounded hidden-state growth that dilutes deeper layers' contributions [25]. We propose *Attention Residuals (AttnRes)*, which replaces this fixed accumulation with softmax attention over preceding layer outputs, allowing each layer to selectively aggregate earlier representations with learned, input-dependent weights. Yet attending over a growing set of preceding outputs is memory-bound on modern accelerators, making the communication cost a practical bottleneck. We address this with *Block AttnRes*, which partitions layers into blocks and attends over block-level representations, reducing per-token memory access from $O(Ld)$ to a fixed $O(Nd)$ while retaining most of the expressiveness. Together with cache-based pipeline communication and a two-phase attention algorithm, it serves as a drop-in replacement for standard residual connections with negligible overhead.

We integrate AttnRes into the Kimi Linear architecture [63] (48A3B) and pretrain on 1.4T tokens. AttnRes mitigates the PreNorm dilution problem, yielding more uniform output magnitudes and gradient distribution across depth. Scaling-law experiments confirm that the improvement is consistent across model sizes, and ablations validate the benefit of content-dependent depth-wise selection.

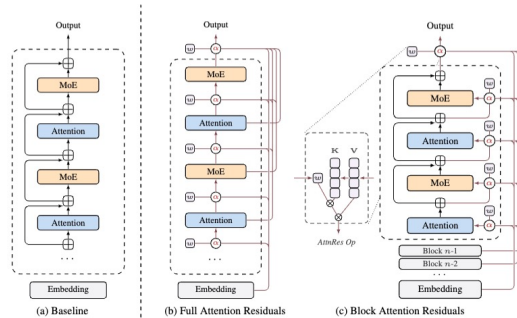
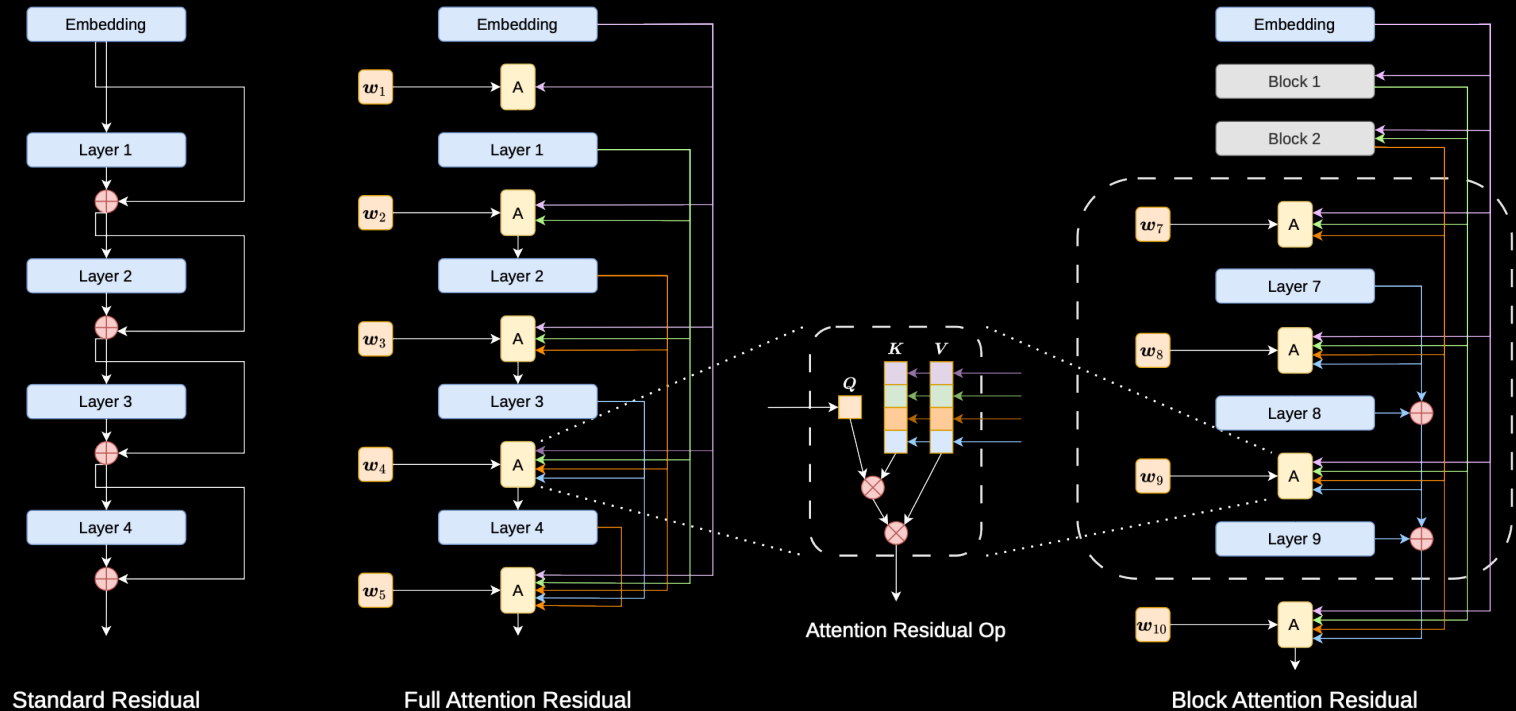


Figure 1: Overview of Attention Residuals. (a) Baseline: standard residual connections with uniform additive accumulation. (b) Full AttnRes: each layer selectively aggregates all previous layer outputs via learned attention weights. (c) Block AttnRes: layers are grouped into blocks, reducing memory from $O(Ld)$ to $O(Nd)$.

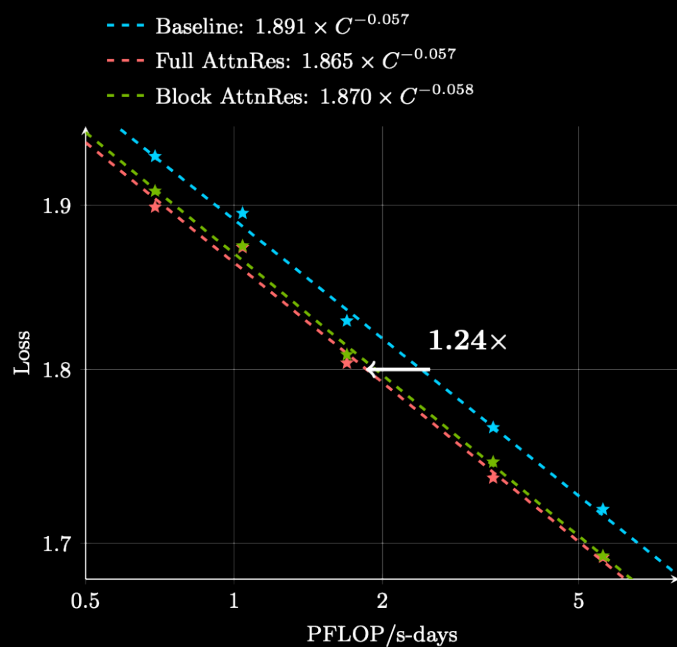


Standard Residual

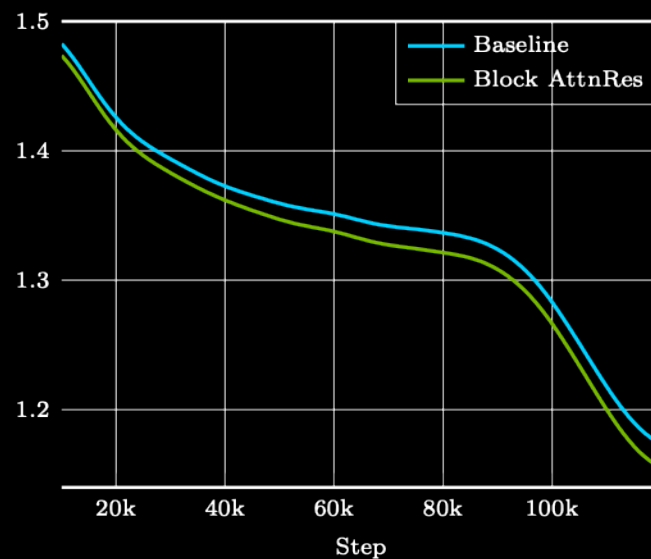
Full Attention Residual

Block Attention Residual

Pretraining Performance of Attention Residuals



Scaling law



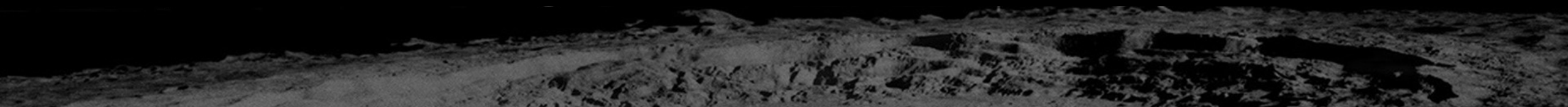
Validation loss

		Baseline	AttnRes	
General	MMLU	73.5	74.6	
	MMLU-Pro	52.2	52.2	
	GPQA-Diamond	36.9	44.4	+7.5
	BBH	76.3	78.0	
	ARC-Challenge	64.6	65.7	
	HellaSwag	83.2	83.4	
Math & Code	TriviaQA	69.9	71.8	
	GSM8K	81.7	82.4	
	MGSM	64.9	66.1	
	Math	53.5	57.1	+3.6
	CMath	84.7	85.1	
	HumanEval	59.1	62.2	+3.1
Chinese	MBPP	72.0	73.9	
	CMMLU	82.0	82.9	
	C-Eval	79.6	82.5	

Benchmark results

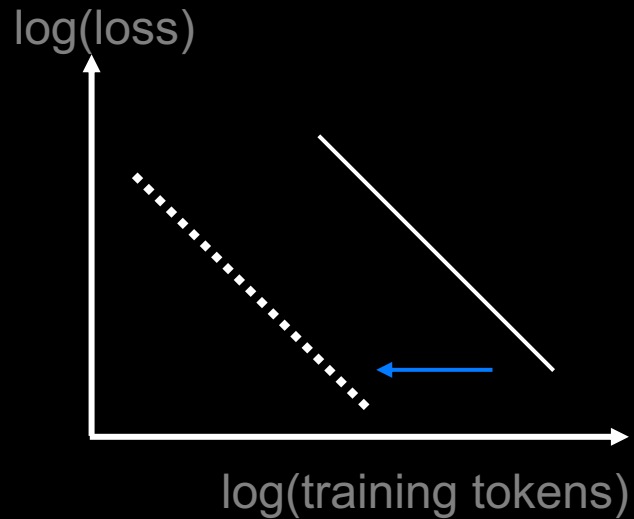
Open Models, New Architectures

- ❑ Adam was invented 11+ years ago → we scaled and open-sourced MuonClip
- ❑ Attention was invented 8+ years ago → we scaled and open-sourced Kimi Linear
- ❑ Residual connections was invented 10+ years ago → we scaled and open-sourced Attention Residuals

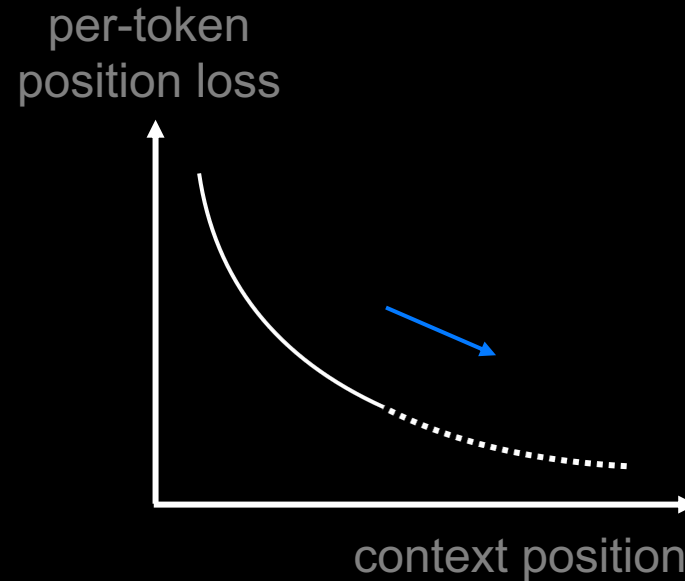


Keep Scaling Kimi's Open Models

Token Efficiency



Long Context



Agent Swarms

